

Security by design

Diapositives complètes — Dr. Zakaria Sawadogo, École Polytechnique de Ouagadougou

Chapitre 1 — Introduction : principes de security by design et privacy by design

Security by design

DR. ZAKARIA SAWADOGO — ÉCOLE POLYTECHNIQUE DE OUAGADOUGOU

1. Définition

Security by design désigne l'approche consistant à intégrer les exigences de sécurité **dès la phase de conception** d'un système, plutôt que de les ajouter a posteriori sous forme de correctifs après un incident ou un audit. C'est un changement de posture : la sécurité devient une contrainte de conception au même titre que la performance ou l'ergonomie, pas une couche additionnelle optionnelle.

2. Pourquoi corriger après coup coûte plus cher

Le coût de correction d'une faille de conception augmente très fortement à mesure que le projet avance : une faille identifiée en phase de conception coûte typiquement un ordre de grandeur de moins à corriger qu'une faille découverte en production, en particulier si elle nécessite de repenser une architecture déjà déployée et adoptée par des utilisateurs. Cette observation motive l'intégration précoce de la sécurité (« shift left »), reprise au chapitre 5 (DevSecOps).

3. Security by design vs sécurité périmétrique

Un modèle de sécurité historique reposait sur une **frontière protégée** (pare-feu, périmètre réseau) protégeant un intérieur considéré comme de confiance. Ce modèle s'avère fragile dès qu'un attaquant franchit le périmètre (par hameçonnage, compromission d'un poste, prestataire tiers compromis) : à l'intérieur, peu de contrôles supplémentaires ralentissent sa progression. *Security by design* pousse à concevoir des systèmes résilients même en présence d'une compromission partielle, préfigurant le modèle zero trust (chapitre 4).

4. Les grands principes, aperçu

Ce module détaille au chapitre 3 une liste de principes de conception sécurisée reconnus (moindre privilège, défense en profondeur, échec sécurisé, séparation des tâches, simplicité de conception, médiation complète), formalisés notamment par Saltzer et Schroeder dès 1975 et toujours d'actualité.

5. Privacy by design

Concept complémentaire, formalisé par Ann Cavoukian (fin des années 1990-2000) et aujourd'hui explicitement exigé par plusieurs réglementations de protection des données (ex. article 25 du RGPD, « protection des données dès la conception et par défaut ») : la protection des données personnelles doit être une caractéristique par défaut d'un système, pas une option activable. Ce concept est développé au chapitre 6.

6. Sécurité et fonctionnalité : un arbitrage, pas une opposition

Une idée reçue oppose sécurité et facilité d'usage. Une conception mature cherche plutôt à rendre le **comportement sécurisé le comportement par défaut et le plus simple** pour l'utilisateur (ex. authentification à double facteur activée par défaut plutôt que reléguée à un réglage avancé rarement découvert). Une mesure de sécurité qui dégrade excessivement l'expérience utilisateur est souvent contournée en pratique (mots de passe notés sur un post-it, désactivation d'un contrôle jugé trop contraignant) — un échec de conception, pas seulement un problème de discipline des utilisateurs.

7. Articulation avec le reste du programme

Ce module synthétise et opérationnalise des notions vues ailleurs dans le programme : vulnérabilités identifiées en *Audit organisation et technique*, gestion des risques du module *Gouvernance*, primitives cryptographiques à intégrer correctement (*Cryptographie*). *Security by design* est la discipline qui transforme ces connaissances en pratiques de conception concrètes, en amont du développement.

À retenir

- *Security by design* intègre la sécurité dès la conception, réduisant fortement le coût de correction par rapport à une approche corrective a posteriori.
- Le modèle de sécurité périmétrique seul est insuffisant face à des attaquants capables de franchir la frontière ; une conception résiliente à la compromission partielle est nécessaire.
- Une mesure de sécurité qui dégrade excessivement l'usage est souvent contournée : la sécurité par défaut et simple d'usage est un objectif de conception, pas un vœu pieux.

Chapitre 2 — Modélisation des menaces (threat modeling)

Security by design

DR. ZAKARIA SAWADO — ÉCOLE POLYTECHNIQUE DE OUAGADOUGOU

| 1. Qu'est-ce que le threat modeling ?

Le threat modeling est une démarche structurée visant à identifier, de façon systématique et le plus en amont possible, les menaces plausibles pesant sur un système, afin de concevoir des mesures de sécurité proportionnées avant l'implémentation. C'est l'un des outils centraux de *security by design*.

2. La démarche générale en quatre questions (Adam Shostack)

1. **Sur quoi travaillons-nous ?** (modéliser le système : diagramme de flux de données, composants, frontières de confiance).
2. **Qu'est-ce qui peut mal tourner ?** (identification systématique des menaces).
3. **Que faisons-nous à ce sujet ?** (définir des contre-mesures).
4. **Avons-nous fait du bon travail ?** (valider la démarche, itérer).

3. Diagramme de flux de données (DFD) et frontières de confiance

Avant d'identifier des menaces, il faut représenter le système : processus, flux de données, magasins de données, entités externes, et surtout les **frontières de confiance** (points où le niveau de confiance change, ex. entre un client non authentifié et un serveur, entre une DMZ et un réseau interne). Les menaces se concentrent typiquement sur ces frontières.

4. STRIDE : classification des menaces

Modèle développé par Microsoft, six catégories de menaces à examiner systématiquement pour chaque composant/flux du diagramme :

Lettre	Menace	Propriété de sécurité visée	Exemple
S	Spoofing (usurpation d'identité)	Authenticité	se faire passer pour un autre utilisateur
T	Tampering (falsification)	Intégrité	modifier des données en transit ou au repos
R	Repudiation (répudiation)	Non-répudiation	nier avoir effectué une action, en l'absence de traçabilité
I	Information disclosure (divulcation d'information)	Confidentialité	accéder à des données sans autorisation
D	Denial of Service (déni de service)	Disponibilité	rendre un service indisponible
E	Elevation of Privilege (élévation de privilège)	Autorisation	obtenir des droits supérieurs à ceux accordés

Pour chaque flux/composant du diagramme, on se demande systématiquement : ce composant est-il vulnérable à S ? à T ? etc.

5. DREAD : évaluation de la sévérité

Modèle complémentaire (moins utilisé aujourd'hui de façon isolée, souvent remplacé par une matrice de risque classique ou CVSS) pour scorer une menace identifiée selon cinq critères : **D**amage (dommage potentiel), **R**eproducibility (reproductibilité), **E**xploitability (facilité d'exploitation), **A**ffected users (utilisateurs affectés), **D**iscoverability (facilité de découverte).

6. Arbres d'attaque (attack trees)

Représentation hiérarchique où le nœud racine est l'objectif de l'attaquant (ex. « compromettre le compte administrateur »), décomposé en sous-objectifs (nœuds enfants) reliés par des relations logiques ET/OU, jusqu'à des actions concrètes réalisables. Cette structure aide à visualiser les différents chemins d'attaque possibles et à identifier où une contre-mesure unique bloque plusieurs branches.

7. PASTA et autres méthodologies

D'autres méthodologies existent, orientées différemment : **PASTA** (Process for Attack Simulation and Threat Analysis) adopte une perspective centrée sur le risque métier et simule des scénarios d'attaque complets ; **LINDDUN** est une méthodologie spécifiquement dédiée aux menaces sur la vie privée (à mettre en lien avec le chapitre 6, *privacy by design*).

8. Quand et comment mener un threat modeling

- Idéalement en phase de conception, avant l'écriture du code, et **révisé** à chaque évolution architecturale significative.
- En atelier collaboratif, associant développeurs, architectes et personnel sécurité — le threat modeling gagne à ne pas être l'exercice isolé d'un seul expert sécurité déconnecté de l'équipe de développement.
- Documenté et priorisé, pour alimenter directement le backlog de développement (contre-mesures = user stories/tâches techniques).

À retenir

- Le threat modeling identifie systématiquement les menaces à partir d'une représentation du système et de ses frontières de confiance, avant l'implémentation.
- STRIDE structure l'identification des menaces par catégorie, en miroir des propriétés de sécurité visées.
- Les arbres d'attaque et les méthodologies alternatives (PASTA, LINDDUN) offrent des perspectives complémentaires selon le contexte (risque métier, vie privée).

Chapitre 3 — Principes de conception sécurisée

Security by design

DR. ZAKARIA SAWADO — ÉCOLE POLYTECHNIQUE DE OUAGADOUGOU

1. Moindre privilège (least privilege)

Chaque composant, utilisateur ou processus ne doit disposer que des droits strictement nécessaires à l'accomplissement de sa tâche, ni plus.

Application concrète : un service applicatif se connectant à une base de données ne doit pas utiliser un compte disposant de droits d'administration complets si seules des opérations de lecture/écriture sur des tables spécifiques sont nécessaires. Ce principe limite l'impact d'une compromission (un composant compromis ne peut nuire qu'à hauteur de ses propres privilèges).

2. Défense en profondeur (defense in depth)

Superposer plusieurs couches de contrôles indépendantes, de sorte que la défaillance d'une seule couche ne compromette pas l'ensemble du système. Exemple pour une application web : pare-feu réseau, pare-feu applicatif (WAF), validation des entrées côté serveur, requêtes préparées contre l'injection SQL, chiffrement des données sensibles au repos, journalisation et détection d'anomalies. Aucune couche seule n'est infaillible ; leur combinaison réduit fortement la probabilité qu'une seule faille suffise à compromettre le système entier.

3. Échec sécurisé (fail-safe / fail-secure)

En cas d'erreur ou de défaillance, un système doit basculer vers l'état le plus sûr, pas vers un état permissif. Exemple : si un mécanisme de vérification d'autorisation échoue techniquement (exception, timeout), le comportement par défaut doit être de **refuser** l'accès, jamais de l'accorder par défaut. Une erreur de logique inversant ce principe (`if erreur: autoriser_acces()` au lieu de refuser) est une classe de bug de sécurité récurrente.

4. Médiation complète (complete mediation)

Chaque accès à une ressource protégée doit être vérifié systématiquement, à chaque tentative, sans mise en cache non maîtrisée d'une autorisation antérieure qui pourrait être devenue caduque (ex. droit révoqué entre-temps). Une vérification effectuée une seule fois « à l'entrée » puis considérée valide pour toute la suite d'une session peut devenir obsolète si les droits changent en cours de session.

5. Séparation des tâches (separation of duties)

Répartir des opérations sensibles entre plusieurs personnes ou composants distincts, de sorte qu'aucune entité unique ne puisse, seule, réaliser une action critique de bout en bout sans contrôle croisé. Exemple organisationnel : la personne qui valide un paiement ne doit pas être la même que celle qui l'initie. Exemple technique : séparer les clés de signature de code des systèmes de build automatisés accessibles à de nombreux développeurs.

6. Simplicité de conception (economy of mechanism)

Un système simple est plus facile à analyser, auditer et prouver correct qu'un système complexe. La complexité est l'ennemie de la sécurité : elle multiplie les cas limites, les interactions imprévues entre composants et la surface d'attaque effective. Ce principe justifie de préférer des architectures et des dépendances minimales à des solutions sur-conçues « au cas où ».

| 7. Conception ouverte (open design) — rappel du principe de Kerckhoffs

Rappel direct du module *Cryptographie* : la sécurité d'un système ne doit pas reposer sur le secret de sa conception, mais sur des mécanismes robustes même connus de l'attaquant (clés, secrets d'authentification), le design lui-même pouvant être examiné publiquement.

8. Minimisation de la surface d'attaque

Réduire le nombre de points d'entrée exposés (fonctionnalités, ports réseau, API, dépendances tierces) au strict nécessaire. Chaque fonctionnalité, endpoint ou dépendance supplémentaire est une surface d'attaque potentielle additionnelle à sécuriser et maintenir — un principe qui rejoint la minimisation de complexité du point précédent.

9. Valeurs par défaut sécurisées (secure by default)

Un système livré ou installé doit être sécurisé « dès la sortie de la boîte », sans que l'utilisateur ait à activer explicitement les protections de base (ex. chiffrement activé par défaut, comptes par défaut désactivés, ports non essentiels fermés par défaut). Rappel du chapitre 1 : la sécurité facile et par défaut est plus efficace en pratique que la sécurité techniquement supérieure mais optionnelle.

À retenir

- Ces principes (Saltzer et Schroeder, et compléments ultérieurs) forment un socle de conception réutilisable indépendamment de la technologie employée.
- Ils se combinent : la défense en profondeur suppose l'application cohérente du moindre privilège à chaque couche ; l'échec sécurisé suppose une médiation complète.
- Aucun principe pris isolément ne suffit ; c'est leur application systématique et cohérente à l'échelle d'un système qui constitue une véritable démarche *security by design*.

Chapitre 4 — Architecture sécurisée : segmentation et zero trust

Security by design

DR. ZAKARIA SAWADOGO — ÉCOLE POLYTECHNIQUE DE OUAGADOUGOU

1. Le modèle périmétrique traditionnel et ses limites

Le modèle de sécurité « château fort » repose sur un périmètre (pare-feu) séparant un réseau interne considéré de confiance d'un extérieur considéré hostile. Ce modèle, longtemps dominant, s'avère fragile face à des menaces qui n'ont plus besoin de franchir frontalement le périmètre : télétravail massif, usage du Cloud (données et applications hors du périmètre physique traditionnel), compromission initiale par hameçonnage d'un poste interne, prestataires tiers disposant d'accès internes.

2. Segmentation réseau

Avant même le concept de zero trust, la segmentation réduit l'impact d'une compromission en divisant le réseau en zones distinctes avec un filtrage strict entre elles (rappel du module *Audit organisation et technique*, chapitre 4) : DMZ pour les services exposés, réseau interne cloisonné par fonction ou sensibilité, réseau d'administration isolé du réseau utilisateur. Une segmentation fine limite la **propagation latérale (lateral movement)** d'un attaquant ayant compromis un premier point d'entrée.

3. Principes du modèle Zero Trust

Formalisé notamment par le NIST (SP 800-207), le modèle zero trust part du principe qu'**aucune confiance implicite ne doit être accordée du simple fait qu'une requête provient du réseau interne** : chaque accès doit être authentifié, autorisé et vérifié en continu, indépendamment de sa localisation réseau d'origine.

Principes clés

- **Vérification explicite** : authentifier et autoriser systématiquement, en s'appuyant sur toutes les données disponibles (identité, appareil, localisation, comportement).
- **Moindre privilège appliqué dynamiquement** : accès juste-à-temps et juste-suffisant (*just-in-time, just-enough-access*), plutôt que des droits larges accordés une fois pour toutes.
- **Présomption de compromission** (*assume breach*) : concevoir le système comme si un attaquant était déjà présent quelque part dans l'environnement, en minimisant le rayon d'action possible (segmentation fine, micro-segmentation) et en maximisant la détection.

4. Composants typiques d'une architecture zero trust

Composant	Rôle
Gestion des identités (IAM)	authentification forte (MFA), gestion centralisée des identités et des droits
Politique d'accès contextuelle	décision d'accès basée sur de multiples signaux (identité, posture de l'appareil, localisation, sensibilité de la ressource), pas uniquement sur l'appartenance réseau
Micro-segmentation	contrôle fin des flux, y compris entre charges de travail internes (pas seulement à la frontière du réseau)
Chiffrement systématique	chiffrement des flux, y compris en interne, sans supposer un réseau interne intrinsèquement sûr
Supervision continue	journalisation et détection comportementale permettant de révoquer un accès en cours de session si un comportement anormal est détecté

5. Zero trust n'est pas un produit unique

Zero trust est une architecture et une philosophie de conception, pas un produit acheté sur étagère : sa mise en œuvre combine plusieurs briques technologiques existantes (IAM, micro-segmentation, chiffrement, supervision) orchestrées selon les principes ci-dessus, et s'accompagne nécessairement d'une transformation organisationnelle (revue des processus d'accès, culture de vérification continue).

6. Migration progressive

Une migration complète vers zero trust est rarement un projet « big bang » : elle se conduit généralement de façon incrémentale, en priorisant les actifs les plus critiques (identifiés par la démarche de gestion des risques du module *Gouvernance*), avec des architectures hybrides transitoires combinant éléments périmétriques existants et contrôles zero trust nouveaux.

À retenir

- Le modèle périmétrique traditionnel suppose une confiance implicite au réseau interne, une hypothèse de moins en moins tenable (télétravail, Cloud, menaces internes).
- Le zero trust remplace cette confiance implicite par une vérification systématique et continue de chaque accès, quelle que soit sa provenance.
- La mise en œuvre combine des briques technologiques existantes (IAM, micro-segmentation, chiffrement, supervision) et se conduit généralement de façon progressive et priorisée.

Chapitre 5 — Cycle de développement sécurisé (DevSecOps)

Security by design

DR. ZAKARIA SAWADOGO — ÉCOLE POLYTECHNIQUE DE OUAGADOUGOU

1. Le cycle de développement sécurisé (Secure SDLC)

Intégrer la sécurité à chaque phase du cycle de vie du développement logiciel, plutôt que de la traiter comme une étape de validation finale isolée :

Phase	Activités de sécurité
Exigences	exigences de sécurité explicites, classification de la sensibilité des données traitées
Conception	threat modeling (chapitre 2), revue d'architecture
Développement	bonnes pratiques de codage sécurisé, revue de code, analyse statique (SAST)
Tests	tests de sécurité automatisés, analyse dynamique (DAST), tests d'intrusion
Déploiement	durcissement de la configuration, gestion sécurisée des secrets
Exploitation	supervision, gestion des correctifs, réponse à incident

2. « Shift left » : déplacer la sécurité en amont

Le principe du *shift left* consiste à déplacer les activités de sécurité le plus tôt possible dans le cycle de développement (« vers la gauche » d'une frise chronologique), en cohérence avec l'observation du chapitre 1 : plus une faille est détectée tôt, moins elle coûte à corriger.

3. DevSecOps : intégrer la sécurité dans DevOps

DevOps vise à accélérer et fiabiliser le cycle développement-déploiement par l'automatisation (intégration continue, déploiement continu — CI/CD) et une collaboration étroite entre équipes de développement et d'exploitation. **DevSecOps** étend cette approche en intégrant la sécurité comme responsabilité partagée et automatisée tout au long du pipeline, plutôt que comme une porte de validation manuelle et tardive gérée par une équipe séparée.

Contrôles de sécurité automatisables dans un pipeline CI/CD

Étape du pipeline	Contrôle de sécurité
Commit / pre-commit	détection de secrets codés en dur (clés API, mots de passe) avant qu'ils n'entrent dans l'historique du dépôt
Build	analyse statique du code (SAST)
Dépendances	analyse de composition logicielle (SCA) : vulnérabilités connues dans les bibliothèques tierces utilisées
Tests	tests de sécurité automatisés (cas de test dérivés du threat modeling)
Déploiement en environnement de test	analyse dynamique (DAST) contre l'application en fonctionnement
Image de conteneur	analyse de vulnérabilités de l'image avant publication
Infrastructure as Code	analyse de configuration (détection de règles de pare-feu trop permissives, de stockage mal configuré) avant provisionnement

4. Gestion des secrets

Les identifiants, clés d'API et certificats ne doivent jamais être codés en dur dans le code source ou versionnés dans un dépôt Git (même privé — un dépôt peut devenir public par erreur, ou son historique complet reste accessible à quiconque y a eu accès). Bonnes pratiques : gestionnaires de secrets dédiés (Vault et équivalents), variables d'environnement injectées au déploiement, rotation régulière des secrets, scan automatique des dépôts pour détecter des secrets déjà exposés.

5. Gestion des vulnérabilités des dépendances tierces

La majorité du code d'une application moderne provient de dépendances tierces (bibliothèques open source). Une vulnérabilité découverte dans une dépendance largement utilisée peut affecter un très grand nombre d'applications simultanément (illustré par des incidents majeurs affectant des bibliothèques de journalisation ou de compression largement répandues). L'analyse de composition logicielle (SCA) automatisée en continu, couplée à une politique de mise à jour réactive, est devenue une pratique de sécurité de base incontournable.

6. Culture et organisation

DevSecOps est autant une question de culture que d'outillage : formation des développeurs à la sécurité (« security champions » au sein des équipes de développement), responsabilité partagée plutôt que reportée entièrement sur une équipe sécurité séparée en fin de cycle, et retour d'information rapide (un développeur informé d'une faille immédiatement après son introduction la corrige bien plus efficacement qu'informé plusieurs mois plus tard lors d'un audit externe).

| À retenir

- Le Secure SDLC intègre des activités de sécurité à chaque phase du développement, pas uniquement en validation finale.
- DevSecOps automatise ces contrôles directement dans le pipeline CI/CD (SAST, SCA, DAST, scan de secrets, scan d'infrastructure as code).
- La gestion des secrets et des dépendances tierces vulnérables sont deux enjeux opérationnels majeurs et récurrents du développement logiciel moderne.

Chapitre 6 — Privacy by design

Security by design

DR. ZAKARIA SAWADO — ÉCOLE POLYTECHNIQUE DE OUAGADOUGOU

1. Définition et origine

Le *privacy by design* (protection de la vie privée dès la conception) est un cadre formalisé par Ann Cavoukian, articulé autour de sept principes fondateurs, visant à intégrer la protection des données personnelles comme caractéristique intrinsèque d'un système, dès sa conception, plutôt que comme un ajout réglementaire tardif.

2. Les sept principes fondateurs

1. **Proactif, non réactif ; préventif, non correctif** : anticiper les risques pour la vie privée avant qu'un incident ne survienne.
2. **Protection de la vie privée par défaut** : les paramètres les plus protecteurs doivent être activés sans action requise de l'utilisateur.
3. **Protection intégrée à la conception** : composante structurelle du système, pas une fonctionnalité ajoutée après coup.
4. **Fonctionnalité intégrale (« somme positive », pas somme nulle)** : rechercher des solutions où sécurité/fonctionnalité et vie privée coexistent, plutôt que de présenter systématiquement un arbitrage l'un contre l'autre.
5. **Sécurité de bout en bout** : protection tout au long du cycle de vie de la donnée, de la collecte à la suppression.
6. **Visibilité et transparence** : les pratiques réelles doivent être vérifiables, pas seulement déclarées.
7. **Respect de la vie privée des utilisateurs** : conception centrée sur l'intérêt de la personne concernée, avec des interfaces et paramètres clairs et accessibles.

3. Traduction juridique : article 25 du RGPD

Le RGPD (Règlement Général sur la Protection des Données, Union européenne, largement pris en référence au-delà de sa portée juridique directe) reprend ce principe dans son article 25 : « protection des données dès la conception et par défaut » (*privacy by design and by default*), imposant concrètement au responsable de traitement de mettre en œuvre des mesures techniques et organisationnelles appropriées dès la conception d'un traitement de données personnelles.

4. Principes de minimisation

- **Minimisation de la collecte** : ne collecter que les données strictement nécessaires à la finalité poursuivie, pas « au cas où » elles seraient utiles plus tard.
- **Minimisation de la conservation** : définir une durée de conservation limitée et justifiée, avec suppression ou anonymisation effective au-delà.
- **Minimisation de l'accès** : appliquer le principe du moindre privilège (chapitre 3) spécifiquement aux données personnelles.
- **Minimisation de l'identifiabilité** : privilégier, quand la finalité le permet, des données pseudonymisées ou anonymisées plutôt que directement identifiantes.

5. Techniques de protection de la vie privée

Technique	Principe
Pseudonymisation	remplacer un identifiant direct par un pseudonyme, la ré-identification restant possible avec une information additionnelle gardée séparément
Anonymisation	rendre la ré-identification impossible, y compris par recoupement avec d'autres sources (plus difficile à garantir réellement que ce que l'on croit souvent)
Chiffrement	protège la confidentialité des données au repos et en transit (lien direct avec le module <i>Cryptographie</i>)
Confidentialité différentielle (differential privacy)	ajout de bruit statistique calibré à des résultats d'agrégation pour empêcher la ré-identification d'un individu tout en préservant l'utilité statistique globale
Contrôle d'accès et journalisation	limiter et tracer qui accède à quelles données personnelles

6. Analyse d'impact relative à la protection des données (AIPD/DPIA)

Pour un traitement de données présentant un risque élevé pour les droits des personnes, une analyse d'impact formelle est requise par de nombreuses réglementations : elle documente la finalité, la nécessité et la proportionnalité du traitement, évalue les risques pour les personnes concernées, et identifie les mesures pour les atténuer — une démarche directement apparentée à l'analyse de risque du module *Gouvernance et gestion des risques SI*, appliquée spécifiquement à la vie privée.

7. LINDDUN : threat modeling orienté vie privée

Complément du chapitre 2 : LINDDUN structure l'identification de menaces spécifiques à la vie privée (Linkability, Identifiability, Non-repudiation, Detectability, Disclosure of information, Unawareness, Non-compliance), sur le même principe que STRIDE mais orienté vers les atteintes à la vie privée plutôt que vers les propriétés de sécurité classiques.

À retenir

- Le privacy by design intègre la protection des données personnelles comme caractéristique par défaut d'un système, formalisé juridiquement par l'article 25 du RGPD et des dispositions équivalentes ailleurs.
- La minimisation (collecte, conservation, accès, identifiabilité) est le principe opérationnel le plus directement actionnable en conception.
- LINDDUN transpose la logique de threat modeling (STRIDE) spécifiquement aux risques pour la vie privée, complétant l'analyse de sécurité classique.